

Description:

Delta Generators

In mathematics, the symbols Δ and d are often used to denote the difference between two values. Similarly, differentiation takes the ratio of changes (ie. dy/dx) for a linear relationship. This method can be applied multiple times to create multiple 'levels' of rates of change. (A common example is x (position) $\rightarrow v$ (velocity) $\rightarrow a$ (acceleration)).

Today we will be creating a similar concept. Our function `delta` will take a sequence of `values` and a positive integer `level`, and return a sequence with the 'differences' of the original `values`. (Differences here means strictly $b - a$, eg. $[1, 3, 2] \Rightarrow [2, -1]$) The argument `level` is the 'level' of difference, for example `acceleration` is the 2nd 'level' of difference from `position`. The specific encoding of input and output lists is specified below.

The example below shows three different 'levels' of the same input.

```
input = [1, 2, 4, 7, 11, 16, 22]
[...delta(input, 1)] // [1, 2, 3, 4, 5, 6]
[...delta(input, 2)] // [1, 1, 1, 1, 1]
[...delta(input, 3)] // [0, 0, 0, 0]
```

We do not assume any 'starting value' for the input, so the output for each subsequent level will be one item shorter than the previous (as shown above).

If an *infinite* input is provided, then the output must *also* be infinite.

Input/Output encoding

Input and output can be any iterable, possibly infinite. Possibilities include finite lists and possibly infinite generator objects, but any iterable must be accepted as input and is acceptable as output.

Difference implementation

- `delta` must work for iterables of types `number` and `BigInt`, which support the `-` operator.

Additional Requirements/Notes:

- `delta` must work for inputs which are infinite
- `values` will always be valid, and will always produce consistent classes/types of object
- `level` will always be valid, and $1 \leq \text{level} \leq 400$

Additional examples:

```
function* up() {
  for (let a = 0n, b = 1n; true; [a, b] = [a + b, b + 3n])
    yield a
}
```

```
delta(up(), 1); // 1,4,7,10,13,16,19,22,25,28,...
delta(up(), 2); // 3,3,3,3,3,3,3,...
delta(up(), 3); // 0,0,0,0,0,0,0,...
```

Iterators
Recursion